

Q1.

1. For steps - I implemented the Hamming distance heuristic. This counts the number of positions where the current word differs from the goal word.

```
if node.state[i] != self.goal[i]:  
    differences += 1
```

Since the DOGCAT action only changes one letter per turn, at least one action is required for each mismatched position. Every action costs 1.

If the current word differs from the goal at k positions, then every solution requires at least k actions. The true optimal remaining cost is at least k . The heuristic returns k , so it can not overestimate the remaining cost.

2. For scrabble - I implemented a heuristic that adds up the scrabble values of the goal letters at each position where the letter needed to be changed.

```
scrabble_cost += scrabble_values[self.goal[i]]
```

Every mismatched position must eventually be fixed and since there can only be one fix per turn, each action will have its separate turn and will add up to the minimum cost.

The heuristic will only add the costs of the unavoidable changes needed to get to the goal, so it will not exceed the true optimal cost.

3. For frequency - I implemented a heuristic that combines the number of mismatched positions between the current word and goal, and the rarity of the goal word.

```
if node.state[i] != self.goal[i]:  
    differences += 1  
  
return differences + dictionary[self.goal]
```

Every mismatched position must be fixed, so under frequency every action costs at least 1. Also, if the current word is not the goal, then it must eventually get to the goal and incur the rarity cost.

The heuristic only adds the min number of required actions and the rarity cost. It does not add any other costs for words to get there so it can not overestimate the true optimal remaining cost.

Q2.

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* Cost	A* nodes expanded	Reduction
WHY->ASK	Steps	5	773	5	16	97.93%
WHY->ASK	Scrabble	9	765	9	13	98.30%
WHY->ASK	Frequency	38.8554	808	38.8554	634	21.53%
BEAR->DUCK	Steps	4	691	4	6	99.13%
BEAR->DUCK	Scrabble	11	2981	11	6	99.80%
BEAR->DUCK	Frequency	38.2402	1084	38.2402	302	72.14%

1. Yes, UCS always returned the same optimal cost as A* in each experiment. \
2. For WHY->ASK, A* produced the largest percentage reduction under scrabble cost. It reduces the nodes from 765 to 13. For BEAR->DUCK, scrabble also had the largest percentage reduction, from 2981 to just 6 nodes.
3. For both of the pairs, frequency's cost measure was the least informative. The reductions were only 21.53% and 72.14% for the pairs respectively.
4. The scrabble heuristic was the most effective because it provides more cost-specific information rather than just counting the mismatches in the word. The heuristic accurately estimates the optimal remaining cost at the initial state since it includes the unavoidable costs of inserting goal letters. This helps A* prioritize the paths that correct mismatches without adding unnecessary costs.

Q3. For steps - Yes. Every action costs 1 so reversing the path will not have any
For Scrabble - No. Scrabble actions have different costs per each letter. For example, changing dog->cog costs 3, but changing cog->dog costs 2 because d and c have different cost values.
For Frequency - No. Frequency actions depend on the rarity of the resulting word. If we were to reverse an action, it will change the word, which will change the rarity and therefore the cost will be different.

Q4. GPT-5.6 Sol

“ Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w Costs $1 + \text{Rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost

“

$$h(n) = \begin{cases} 0, & n = g \\ h_0(n) + \text{rarity}(g), & n \neq g \end{cases}$$

2.

$$h(n) = \begin{cases} 0, & n = g \\ \sum_{i=1}^L \mathbf{1}[n_i \neq g_i] + \text{rarity}(g), & n \neq g \end{cases}$$

3. The proposed heuristic is admissible. If the current word differs from the goal in k positions, then at least k actions are needed because every action can only change 1 letter. Frequency is defined at $1 + \text{rarity}(g)$, and each action costs at least 1.

5. For WHY->ASK, the original heuristic cost on frequency was 38.8554. For the proposed heuristic, the cost was also 38.8554. Similar for BEAR->DUCK, they returned the same costs at 38.2402. The number of nodes however were 755 and 634 respectively for the WHY->ASK and 720 and 302 for BEAR->DUCK. So the proposed heuristic reduced the number of nodes expanded for both of the tests. This evidence supports using this proposed heuristic because it reduced the number of nodes expanded, but it does not mean it would be more efficient in every case.